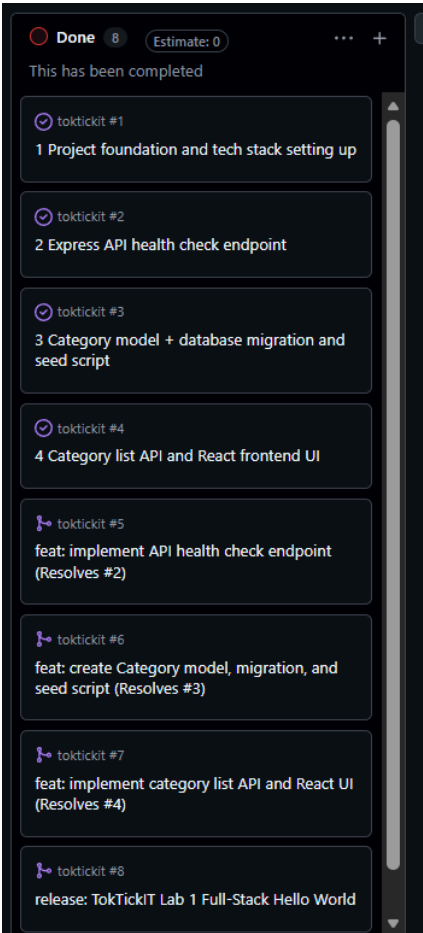
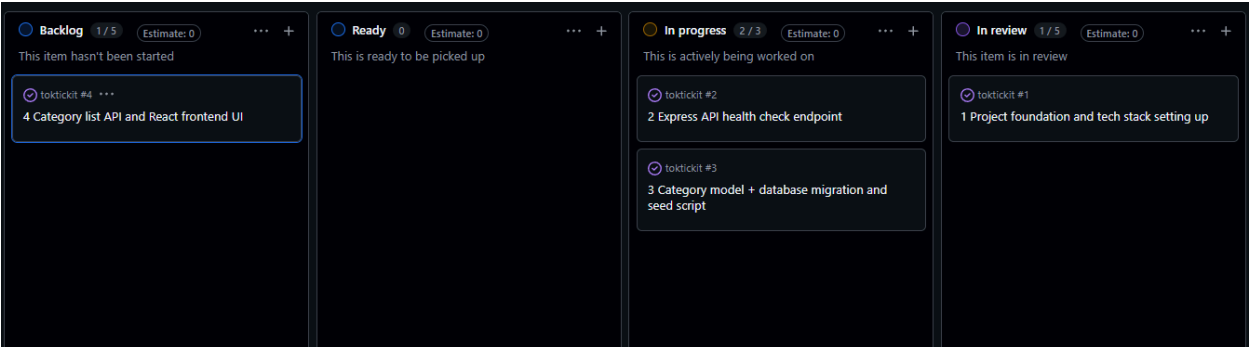


LAB 01 : TokTickIT Full-Stack Hello World Starter

67070503441 Supanut Sopha (ikmie)

Answer Part 1: Git Use with Engineering Workflow

- Git Repository link: <https://github.com/ikmie/toktickit>
- Git project link: <https://github.com/users/ikmie/projects/1>
- GitHub Issues: <https://github.com/ikmie/toktickit/issues>
- Pull request: <https://github.com/ikmie/toktickit/pulls?q=is%3Apr+is%3Aclose>
- issues as Kanban:



- Screenshot of git workflow (commit history and merging to main branch)

Commits

main

All usersAll time

Commits on Aug 10, 2026

Merge pull request #8 from ikmie/lab1-staging
ikmie authored 14 minutes ago
Verified52e3044

Merge pull request #7 from ikmie/feature/4-category-list
ikmie authored 17 minutes ago
Verified25b51b7

Merge pull request #6 from ikmie/feature/3-category-seed
ikmie authored 19 minutes ago
Verified77e4ac7

Merge pull request #5 from ikmie/feature/2-health-check
ikmie authored 22 minutes ago
Verified041ceba

Commits on Aug 8, 2026

feat: implement category list API, React UI, and Vitest UI tests (Issue #4)
ikmie committed 3 days ago
6cbf437

feat: create Category model, migration, and seed script (Issue #3)
ikmie committed 3 days ago
38e979a

feat: implement API health check endpoint and test (Issue #2)
ikmie committed 3 days ago
0ea65cb

Commits on Aug 7, 2026

feat: initial project foundation setup (Issue #1)
ikmie committed 3 days ago
14dad99

- Directory Structure

> client

> docs\lab-01

- ai_use.md
- reviewer.md
- tests.md

> server

- node_modules
- prisma
- src

> tests\lab-01

- categories.test.ts
- health.test.ts
- .env
- .env.example
- package-lock.json
- package.json
- tsconfig.json
- .gitignore
- Lab1_Git_GitHub_CheatSheet.pdf
- Lab1_Glossary.pdf
- Lab1_Labsheet.pdf
- README.md

- README.md

README

67%

Lab 1: Full-Stack Hello World Starter

Repository Structure

```
toktickit/
├── client/      # React + TypeScript + Vite + Bootstrap UI
├── server/      # Express + TypeScript + Prisma API & Tests
│   ├── prisma/  # Prisma schema & seed scripts
│   ├── src/     # Server source code
│   └── tests/    # Supertest API tests
│       └── lab-01/
├── docs/        # Documentation & submission materials
│   └── lab-01/
│       ├── ai_use.md
│       ├── reviewer.md
│       └── tests.md
├── .gitignore
└── README.md
```

Setup Instructions

1. Install Server Dependencies & Setup DB:

```
cd server
npm install
cp .env.example .env
# Ensure PostgreSQL is running and DATABASE_URL in .env is correct
npx prisma migrate dev --name init
npx prisma db seed
```

2. Install Client Dependencies:

```
cd ../client
npm install
```

3. Run Application:

- Start Server: `npm run dev` (in `server/`)
- Start Client: `npm run dev` (in `client/`)

4. Run Tests:

- Server Tests: `npm run test` (in `server/`)
- Client Tests: `npm run test` (in `client/`)

- .gitignore content

main

+

Q

Go to file

T

> client

> docs

> server

.gitignore

README.md

Code

Blame

30 lines (25 loc) · 294 Bytes

```
1  # Dependencies
2  node_modules/
3  .pnpm-store/
4
5  # Environment variables
6  .env
7  .env.local
8  .env.production
9  .env.development.local
10
11 # Build outputs
12 dist/
13 build/
14 out/
15
16 # Logs
17 npm-debug.log*
18 yarn-debug.log*
19 yarn-error.log*
20 logs/
21
22 # IDE / Editor
23 .vscode/
24 .idea/
25 *.swp
26 *.swo
27
28 # OS files
29 .DS_Store
30 Thumbs.db
```

- PR Review Evidence rendered version of docs/lab-01/reviewer.md

Lab 01 - Peer Reviewer Record

Reviewer Details

- Reviewer Name: Wichitchai Suwanno
- Student ID: 67070503439
- GitHub Username: SinghLemonH

1. PR #1 - Project Foundation

◦ Branch: feature/1-project-foundation -> lab1-staging

◦ PR Link: N/A (Initial commit 14dad99 directly initialized lab1-staging as base repository setup)

◦ Peer Reviewer Comment: Project foundation and initial repository structure verified.

◦ Author Response: Initialized lab1-staging base branch with project foundation setup.

2. PR #2 - API Health Check

◦ Branch: feature/2-health-check -> lab1-staging

◦ PR Link: #5

◦ Peer Reviewer Comment: LGTM! Health check endpoint correctly returns status OK and timestamp.

◦ Author Response: Merged into lab1-staging .

3. PR #3 - Category Seed

◦ Branch: feature/3-category-seed -> lab1-staging

◦ PR Link: #6

◦ Peer Reviewer Comment: Category model, database migration, and seed scripts look solid.

◦ Author Response: Merged into lab1-staging .

4. PR #4 - Category List

◦ Branch: feature/4-category-list -> lab1-staging

◦ PR Link: #7

◦ Peer Reviewer Comment: Category list API endpoints and React UI verified with automated tests.

◦ Author Response: Merged into lab1-staging .

Answer Part 2: Tests

- docs/lab-01/tests.md (rendered version)

Lab 01 - Automated Tests Record

Test File (tests/lab-01/)	Tool	Test Description	Status
API-01 (health.test.ts)	Supertest	Health endpoint returns 200 and expected JSON	Passed
API-02 (categories.test.ts)	Supertest	Categories endpoint returns the four seeded categories	Passed
UI-01 (App.test.tsx)	Vitest	TokTickIT heading renders	Passed
UI-02 (App.test.tsx)	Vitest	Loading state changes to category list	Passed
UI-03 (App.test.tsx)	Vitest	API failure displays a useful error message	Passed

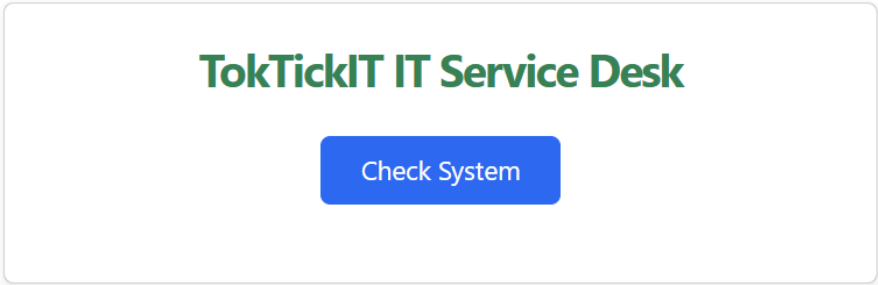
Answer Part 3: AI Use and Reflection

Lab 01 - AI Use & Reflection

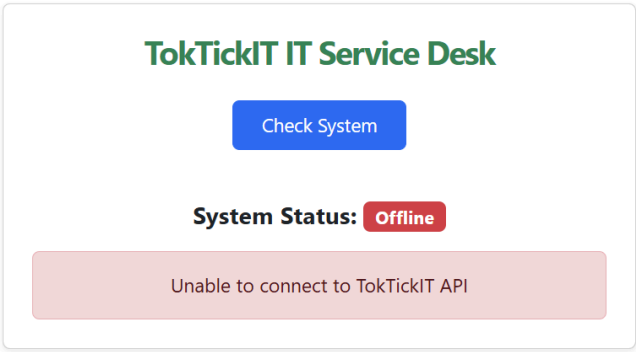
I used the Antigravity coding agent through my Google Cloud Platform account. I mainly used Gemini 3.6 Flash as the LLM with a thinking level of High.

Prompt Name	Actual Prompt Text	Reflection
Plan Lab 1 Implementation	Read the enclosed TokTickIT Lab 1 requirements. Summarize the four GitHub Issues, their dependencies, required outputs, and required automated tests. Propose an implementation order, but do not write code yet.	The agent created a detailed implementation plan covering all 4 issues and dependency ordering.
Set Up Full-Stack Project	Setup the TokTickIT project tech stack as given in Lab 1 using React, TypeScript, Vite, and Bootstrap for the frontend, and Node.js, Express, and TypeScript for the backend. Configure PostgreSQL and Prisma. Use the required folder structure.	The agent initialized client and server applications matching the required structure.
Implement Health Check	Add GET /api/health to the existing Express backend and add Supertest test.	The agent implemented the health check route and generated an automated Supertest test suite to verify the 200 OK response.
Implement Category Seed	Create the Prisma Category model, migration, and seed script for 4 categories.	The agent created the Category model in Prisma schema, executed PostgreSQL migrations, and implemented the seed script for the 4 required categories.
Build Check System UI	Create Bootstrap-based React UI with Check System button, loading state, status display, and category list.	The agent built the React component with Bootstrap styling, managing loading spinners, health status badges, and dynamic category rendering.
Review Lab 1 Work	Review final implementation against acceptance criteria.	The agent conducted a systematic review against all acceptance criteria, ensuring both automated tests passed and full-stack API integration was complete.

Answer Part 4: App demo



- Failure Case after click [Check System]



- Success Case after click [Check System]

